

SQL ANALYSIS (All 20 Queries)

ROUTE ANALYSIS

1. Highest passenger traffic

01create schema 02table creation 03import success 04Indexing q1 x q2 q3 q4 q5 q6

Limit to 1000 rows

```
1 SELECT From_Station, To_Station, SUM(Passengers) AS total_passengers
2 FROM metro_trips
3 GROUP BY From_Station, To_Station
4 ORDER BY total_passengers DESC;
```

Result Grid

	From_Station	To_Station	total_passengers
▶	Rajiv Chowk	Chandni Chowk	1758
	Rajiv Chowk	Noida City Centre	1605
	Rajiv Chowk	Old Delhi	1591
	Rajiv Chowk	Kirti Nagar	1553
	Rajiv Chowk	Janakpuri West	1527
	Rajiv Chowk	Jasola Vihar	1520
	Rajiv Chowk	AIIMS	1504
	Rajiv Chowk	Netaji Subhash Place	1495
	Rajiv Chowk	Kalkaji Mandir	1475
	Rajiv Chowk	Hauz Khas	1433
	Rajiv Chowk	Central Secretariat	1418

Result Grid
Form Editor
Field Types

2. Highest revenue routes

01create schema 02table creation 03import success 04Indexing q1 q2 x q3 q4 q5 q6

Limit to 1000 rows

```
1 SELECT From_Station, To_Station, SUM(Fare) AS total_revenue
2 FROM metro_trips
3 GROUP BY From_Station, To_Station
4 ORDER BY total_revenue DESC;
```

Result Grid

	From_Station	To_Station	total_revenue
▶	Rajiv Chowk	Chandni Chowk	9493.74000453949
	Rajiv Chowk	Hauz Khas	8257.649970054626
	Rajiv Chowk	Kalkaji Mandir	8203.740000724792
	Rajiv Chowk	Old Delhi	8157.980032920837
	Rajiv Chowk	Dilshad Garden	8097.830004692078
	Rajiv Chowk	AIIMS	8082.530006408691
	Rajiv Chowk	Noida City Centre	7972.0999574661255
	Rajiv Chowk	Laxmi Nagar	7795.990026473999
	Rajiv Chowk	Kirti Nagar	7720.310024261475
	Rajiv Chowk	Jasola Vihar	7592.180005073547
	Rajiv Chowk	Janakpuri West	7470.990058898926

Result Grid
Form Editor
Field Types

3. Average fare per route

```
01create schema 02table creation 03import success 04Indexing q1 q2 q3 x q4 q5 q6
1 SELECT From_Station, To_Station, AVG(Fare) AS avg_fare
2 FROM metro_trips
3 GROUP BY From_Station, To_Station;
```

From_Station	To_Station	avg_fare
Aiims	Aiims	108.06333414713542
Aiims	Barakhamba Road	126.24399948120117
Aiims	Central Secretariat	87.59760055541992
Aiims	Chandni Chowk	97.53500032424927
Aiims	Dilshad Garden	128.5229418137494
Aiims	Hauz Khas	98.76476169767834
Aiims	Inderlok	106.63055573569403
Aiims	Janakpuri West	90.25066579182943
Aiims	Jasola Vihar	100.5936356457797
Aiims	Kalkaji Mandir	104.46571350097656
Aiims	Kashmere Gate	82.19391275488812

4. Longest routes

```
01create schema 02table creation 03import success 04Indexing q1 q2 q3 q4 x q5 q6
1 SELECT From_Station, To_Station, MAX(Distance_km) AS max_distance
2 FROM metro_trips
3 GROUP BY From_Station, To_Station
4 ORDER BY max_distance DESC;
```

From_Station	To_Station	max_distance
New Delhi	Mandi House	45.69
Hauz Khas	Dilshad Garden	43.94
Model Town	Mandi House	43.38
Janakpuri West	Laxmi Nagar	42.49
Mandi House	Hauz Khas	40.23
Rajiv Chowk	Kashmere Gate	37.47
Laxmi Nagar	Shivaji Park	37.2
Aiims	Central Secretariat	36.68
New Delhi	Laxmi Nagar	36.41
Chandni Chowk	Rajiv Chowk	36.34
Aiims	Dilshad Garden	36.08

STATION ANALYSIS

5. Most departures

The screenshot shows a database query editor with a tab labeled 'q5'. The SQL query is as follows:

```
1 SELECT From_Station, COUNT(*) AS departures
2 FROM metro_trips
3 GROUP BY From_Station
4 ORDER BY departures DESC;
```

Below the query editor, the 'Result Grid' is displayed, showing the results of the query. The table has two columns: 'From_Station' and 'departures'.

From_Station	departures
Rajiv Chowk	1694
Noida City Centre	1237
New Delhi	838
Mandi House	832
Chandni Chowk	450
Inderlok	447
Kashmere Gate	425
Model Town	425
Aiims	423
Laxmi Nagar	422
Jasola Vihar	412

6. Most passengers received

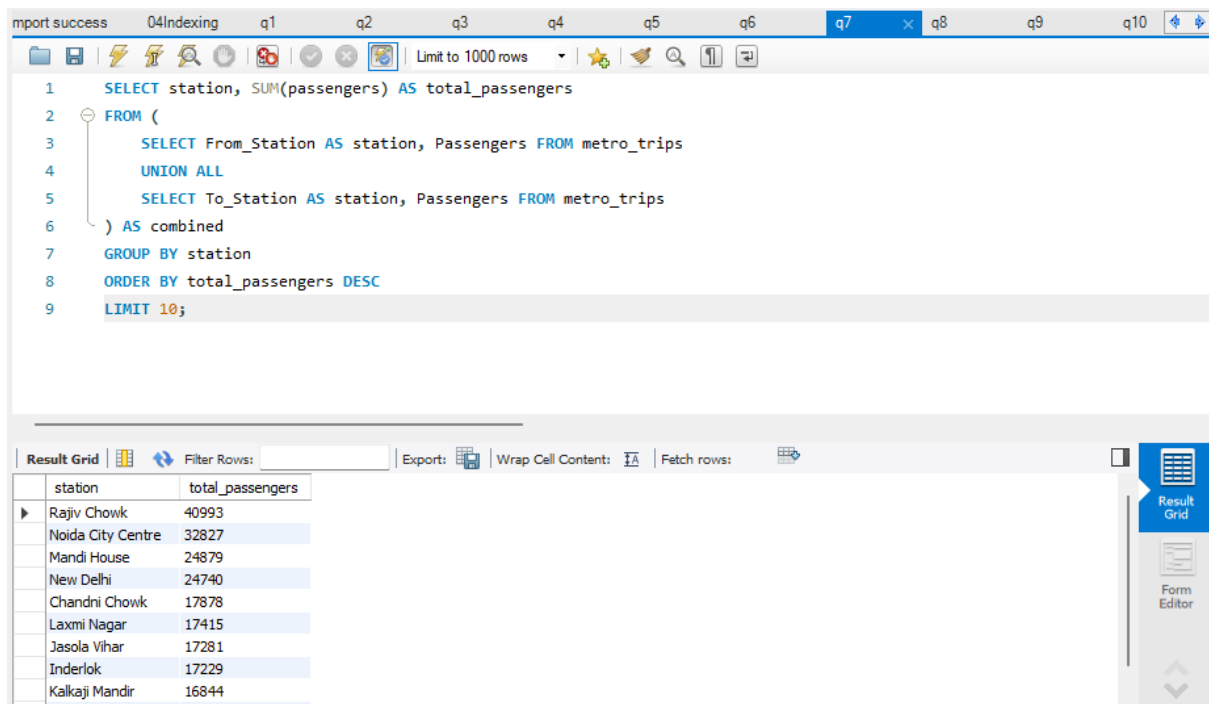
The screenshot shows a database query editor with a tab labeled 'q6'. The SQL query is as follows:

```
1 SELECT To_Station, SUM(Passengers) AS total_passengers
2 FROM metro_trips
3 GROUP BY To_Station
4 ORDER BY total_passengers DESC;
```

Below the query editor, the 'Result Grid' is displayed, showing the results of the query. The table has two columns: 'To_Station' and 'total_passengers'.

To_Station	total_passengers
Jasola Vihar	9014
Laxmi Nagar	9009
Chandni Chowk	8810
Barakhamba Road	8705
Rajouri Garden	8657
Kalkaji Mandir	8649
Netaji Subhash Place	8500
Noida City Centre	8445
Inderlok	8348
Aiims	8233
Dilshad Garden	8231

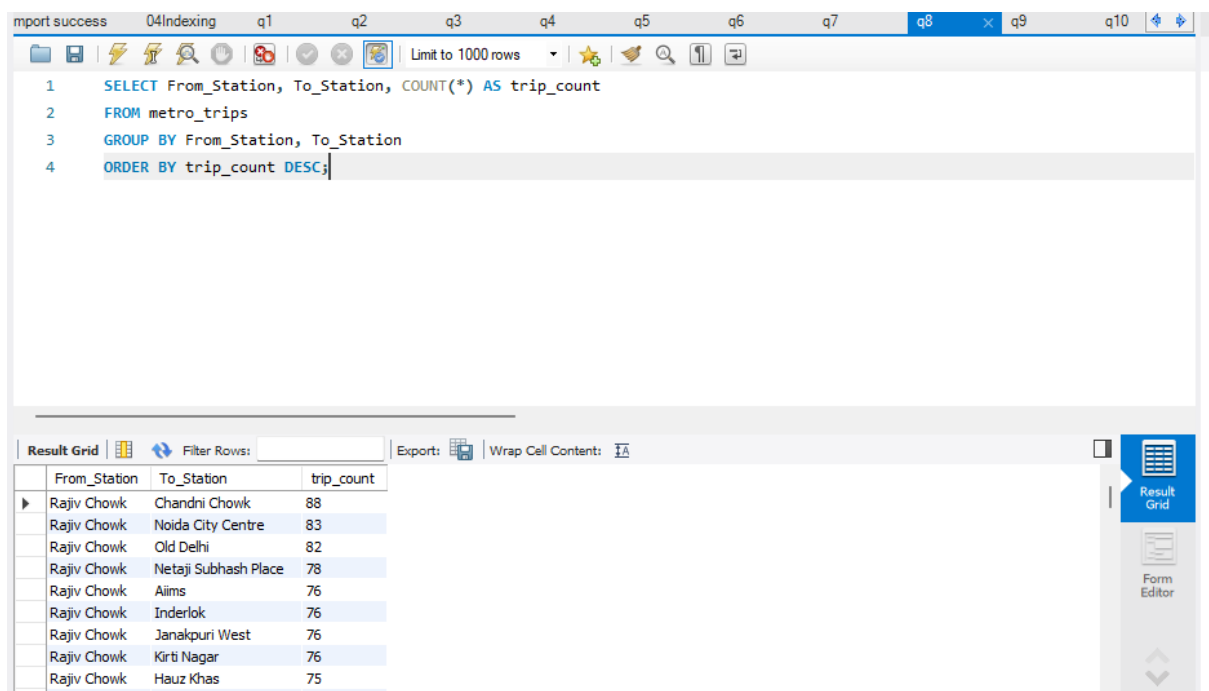
7. Top 10 stations (combined)



```
1 SELECT station, SUM(passengers) AS total_passengers
2 FROM (
3     SELECT From_Station AS station, Passengers FROM metro_trips
4     UNION ALL
5     SELECT To_Station AS station, Passengers FROM metro_trips
6 ) AS combined
7 GROUP BY station
8 ORDER BY total_passengers DESC
9 LIMIT 10;
```

station	total_passengers
Rajiv Chowk	40993
Noida City Centre	32827
Mandi House	24879
New Delhi	24740
Chandni Chowk	17878
Laxmi Nagar	17415
Jasola Vihar	17281
Inderlok	17229
Kalkaji Mandir	16844

8. Most frequent station pairs



```
1 SELECT From_Station, To_Station, COUNT(*) AS trip_count
2 FROM metro_trips
3 GROUP BY From_Station, To_Station
4 ORDER BY trip_count DESC;
```

From_Station	To_Station	trip_count
Rajiv Chowk	Chandni Chowk	88
Rajiv Chowk	Noida City Centre	83
Rajiv Chowk	Old Delhi	82
Rajiv Chowk	Netaji Subhash Place	78
Rajiv Chowk	Aiims	76
Rajiv Chowk	Inderlok	76
Rajiv Chowk	Janakpuri West	76
Rajiv Chowk	Kirti Nagar	76
Rajiv Chowk	Hauz Khas	75

REVENUE ANALYSIS

9. Total revenue

The screenshot shows a data query interface with a toolbar at the top containing icons for file operations, search, and execution. The query editor displays the following SQL code:

```
1 SELECT SUM(Fare) AS total_revenue
2 FROM metro_trips;
```

Below the query editor, the 'Result Grid' tab is active, showing a single row of results:

total_revenue
1043861.3202486038

On the right side of the interface, there are buttons for 'Result Grid' and 'Form Editor'.

10. Average fare per trip

The screenshot shows a data query interface with a toolbar at the top containing icons for file operations, search, and execution. The query editor displays the following SQL code:

```
1 SELECT AVG(Fare) AS avg_fare
2 FROM metro_trips;
```

Below the query editor, the 'Result Grid' tab is active, showing a single row of results:

avg_fare
104.38613202486039

On the right side of the interface, there are buttons for 'Result Grid' and 'Form Editor'.

11. Revenue per km

q7 q8 q9 q10 **q11** q12 q13 q14 q15 q16 q17 q18

Limit to 1000 rows

```
1 SELECT From_Station, To_Station,  
2 SUM(Fare)/SUM(Distance_km) AS revenue_per_km  
3 FROM metro_trips  
4 GROUP BY From_Station, To_Station  
5 ORDER BY revenue_per_km DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	From_Station	To_Station	revenue_per_km
▶	Dilshad Garden	Dilshad Garden	255.43331402805194
	Central Secretariat	Pragati Maidan	47.08540038440141
	Kalkaji Mandir	Barakhamba Road	45.300688455124345
	Laxmi Nagar	Laxmi Nagar	42.599323126032125
	Aiims	Aiims	40.473158904792555
	Inderlok	Dilshad Garden	36.45426247369123
	Rajouri Garden	Dilshad Garden	34.527857702273984
	Inderlok	Noida City Centre	33.756818130150656
	Jasola Vihar	Jasola Vihar	33.67142704554966

Result Grid | Form Editor | Field Types

12. Revenue by ticket type

q5 q6 q7 q8 q9 q10 q11 **q12** q13 q14 q15 q16

Limit to 10000 rows

```
1 SELECT Ticket_Type, SUM(Fare) AS total_revenue  
2 FROM metro_trips  
3 GROUP BY Ticket_Type  
4 ORDER BY total_revenue DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	Ticket_Type	total_revenue
▶	Tourist Card	506804.7798013687
	Single	266358.73046875
	Smart Card	259083.6999912262
	Unknown	11614.109987258911

Result Grid | Form Editor | Field Types

PASSENGER ANALYSIS

13. Avg passengers per trip

Limit to 10000 rows

```
1 SELECT AVG(Passengers) AS avg_passengers
2 FROM metro_trips;
```

Result Grid

avg_passengers
19.8378

Filter Rows: Export: Wrap Cell Content: Form Editor Field Types

14. Highest passenger trips

q5 q6 q7 q8 q9 q10 q11 q12 q13 q14 x q15 q16

Limit to 10000 rows

```
1 SELECT *
2 FROM metro_trips
3 ORDER BY Passengers DESC
4 LIMIT 10;
```

Result Grid

TripID	Date	From_Station	To_Station	Distance_km	Fare	Cost_per_passenger	Passengers	Ticket_Type	Remarks
93442	2023-11-04	Mandi House	Laxmi Nagar	12.55	172.53	65.7	37	Smart Card	off-peak
140529	2022-03-01	New Delhi	Noida City Centre	9.73	94.57	91.66	36	Tourist Card	maintenance
78615	2022-06-13	Jasola Vihar	Central Secretariat	7.67	189.12	8.62	36	Tourist Card	maintenance
43161	2024-02-11	New Delhi	Punjabi Bagh	6.03	129.16	95.29	36	Single	Nil
8614	2022-04-27	Jasola Vihar	Rajouri Garden	1.51	20.97	11.09	35	Tourist Card	Nil
27146	2023-11-26	Mandi House	Kirti Nagar	2.14	127.96	24.18	35	Single	maintenance
92107	2024-07-01	Dilshad Garden	Janakpuri West	3.98	38.89	42.49	35	Tourist Card	festival
52251	2023-07-10	New Delhi	Janakpuri West	1.08	136.09	99.46	35	Smart Card	peak
48752	2024-04-01	Chandni Chowk	Pragati Maidan	6.33	169.94	83.15	35	Tourist Card	Nil
87385	2024-10-15	Central Secretariat	Chandni Chowk	2.11	172.91	43.96	35	Single	maintenance
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Filter Rows: Edit: Export/Import: Wrap Cell Content: Fetch rows: Form Editor Field Types

15. Passenger distribution by ticket

```
q5 q6 q7 q8 q9 q10 q11 q12 q13 q14 q15 x q16
SELECT Ticket_Type, SUM(Passengers) AS total_passengers
FROM metro_trips
GROUP BY Ticket_Type;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

Ticket_Type	total_passengers
Single	50357
Smart Card	49535
Tourist Card	96239
Unknown	2247

Result Grid
Form Editor
Field Types

16. Passenger count per station

```
q5 q6 q7 q8 q9 q10 q11 q12 q13 q14 q15 q16 x
SELECT station, SUM(passengers) AS total_passengers
FROM (
  SELECT From_Station AS station, Passengers FROM metro_trips
  UNION ALL
  SELECT To_Station AS station, Passengers FROM metro_trips
) AS combined
GROUP BY station;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

station	total_passengers
Chandni Chowk	17878
Noida City Centre	32827
Rajiv Chowk	40993
New Delhi	24740
Aiims	16599
Mandi House	24879
Kalkaji Mandir	16844
Kashmere Gate	16509
Rajouri Garden	16174
Model Town	16242
Laxmi Nagar	17415

Result Grid
Form Editor
Field Types

TRAVEL PATTERN ANALYSIS

17. Trips by condition

```
q10 q11 q12 q13 q14 q15 q16 q17 x q18 q19 q20
SELECT Remarks, COUNT(*) AS trip_count
FROM metro_trips
GROUP BY Remarks;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

Remarks	trip_count
festival	1673
maintenance	1649
Nil	1754
off-peak	1655
peak	1618
weekend	1651

Result Grid
Form Editor
Field Types

18. Revenue by condition

```
q10 q11 q12 q13 q14 q15 q16 q17 q18 x q19 q20
SELECT Remarks, SUM(Fare) AS total_revenue
FROM metro_trips
GROUP BY Remarks
ORDER BY total_revenue DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

Remarks	total_revenue
Nil	182216.1699924469
festival	177420.34010219574
weekend	174009.2900800705
off-peak	172776.0399980545
peak	169052.76992321014
maintenance	168386.71015262604

Result Grid
Form Editor
Field Types

19. Monthly passenger trend

The screenshot shows a SQL query editor with a toolbar at the top containing icons for file operations, execution, and settings. The query is as follows:

```
1 SELECT MONTH(Date) AS month, SUM(Passengers) AS total_passengers
2 FROM metro_trips
3 GROUP BY month
4 ORDER BY month;
```

Below the query editor is the 'Result Grid' section, which includes a 'Filter Rows' input field, an 'Export' button, and a 'Wrap Cell Content' checkbox. The result grid displays the following data:

	month	total_passengers
1	1	17334
2	2	15859
3	3	16251
4	4	17199
5	5	17072
6	6	15680
7	7	16943
8	8	16179
9	9	16891

On the right side of the interface, there are buttons for 'Result Grid', 'Form Editor', and 'Field Types'.

20. Avg passengers by condition

The screenshot shows a SQL query editor with a toolbar at the top. The query is as follows:

```
1 SELECT Remarks, AVG(Passengers) AS avg_passengers
2 FROM metro_trips
3 GROUP BY Remarks
4 ORDER BY avg_passengers DESC;
```

Below the query editor is the 'Result Grid' section, which includes a 'Filter Rows' input field, an 'Export' button, and a 'Wrap Cell Content' checkbox. The result grid displays the following data:

	Remarks	avg_passengers
1	off-peak	20.0882
2	Nil	19.8358
3	festival	19.8165
4	maintenance	19.8102
5	weekend	19.7456
6	peak	19.7281

On the right side of the interface, there are buttons for 'Result Grid', 'Form Editor', and 'Field Types'.